

Rapport exercice VI

I Présentation de l'exercice

Le but de l'exercice était de récupérer une implantation déjà faite de l'algorithme Dijkstra, et de chercher à l'optimiser en parallélisant le programme.

J'ai fait le choix de réécrire l'implantation en C++, car je voulais expérimenter l'utilisation d'OpenMp en C++, je ne pense pas que cela change de manière significative les temps enregistrés, je n'ai pas utilisé de fonctionnalités haut niveau de la librairie standard de C++, mis à part les vecteurs, qui sont alloués préalablement, ce qui permet d'avoir une allocation mémoire similaire à un simple malloc en C.

II Calcul du temps

Afin de calculer les temps d'exécution, je me suis aidé d'un script Python, et de la sortie standard du programme en c.

Toutes les données sont donc automatiquement compilées dans un fichier Json.

III Résultats

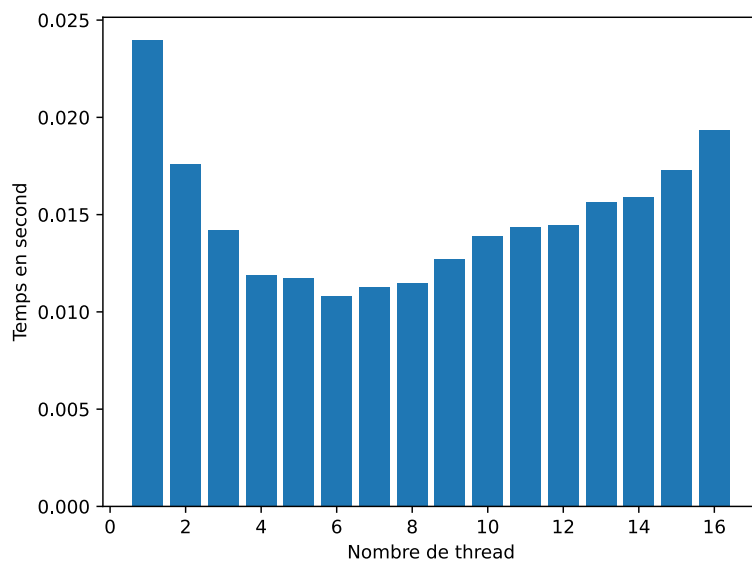
Voici les résultats en fonction de chacune des images données en test, avec un nombre de threads allant de 1 à 16.

III.1 Temps pour le graphe représentant Rome

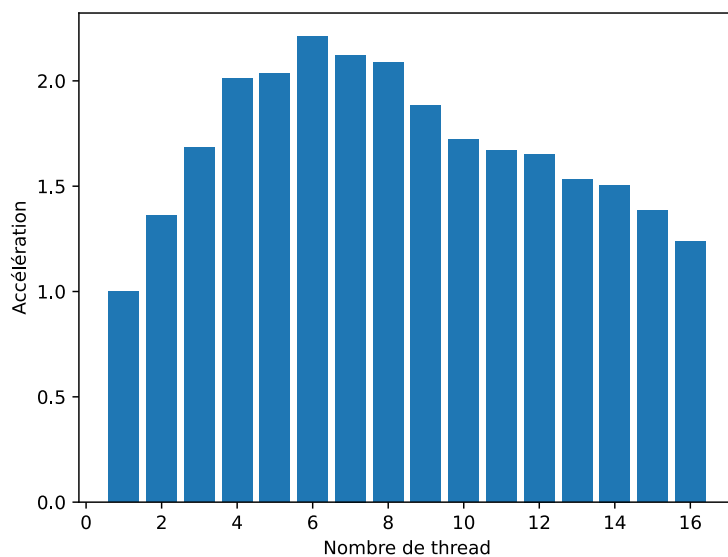
Ce graphe contient 3353 nœuds et 8870 arrêtes, voici les temps obtenus deçus :

Nombre de threads	Temps pris
1	0.023947s
2	0.017601s
3	0.01421s
4	0.011903s
5	0.011759s
6	0.010828s
7	0.011282s
8	0.011478s

Nombre de thread	Temps pris
9	0.01272s
10	0.013897s
11	0.014354s
12	0.014485s
13	0.015648s
14	0.015897s
15	0.01729s
16	0.019332s



On peut observer une bonne évolution du temps, ce dernier semble baisser de moitié, la parallélisation est donc efficace. Regardons cela plus concrètement avec un graphique représentant l'accélération :



L'accélération maximale est d'environ 2.3, aux alentours de six threads. Après six threads, le temps redevient croissant, cela doit être dû à la synchronisation entre les threads, en effet, l'implantation utilise un bon nombre de barrières de synchronisations.

III.2 Temps pour le graphe représentant New York

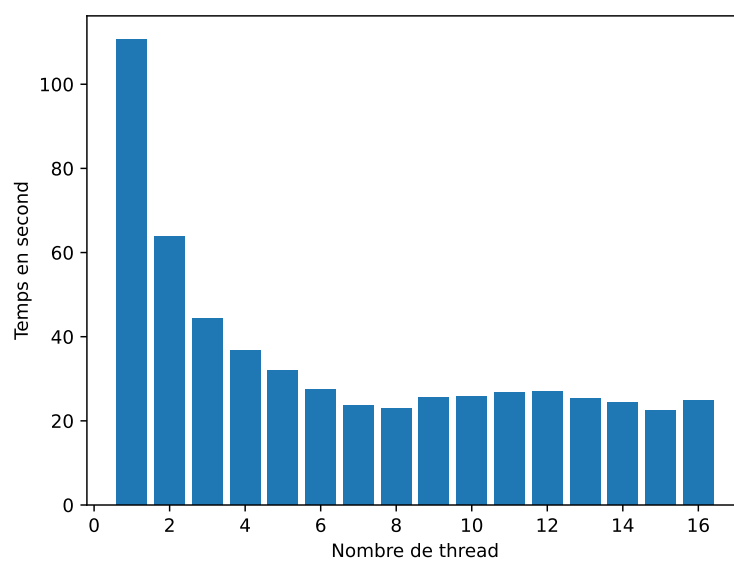
Le deuxième graphe (New York) est de tailles conséquentes, 264346 nœuds et 733846 arrêtes, voici les résultats obtenus :

Nombre de thread	Temps pris
1	110.713s

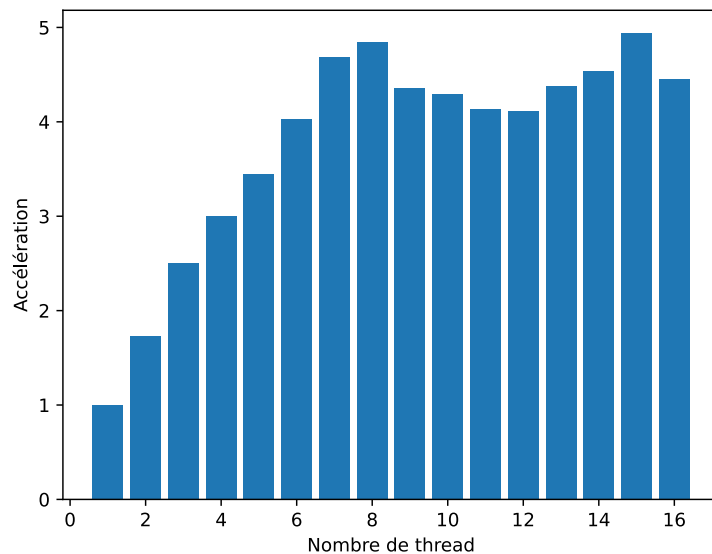
Nombre de thread	Temps pris
2	63.9042s

Nombre de thread	Temps pris
3	44.2873s
4	36.8271s
5	32.0795s
6	27.4911s
7	23.6286s
8	22.8641s

Nombre de thread	Temps pris
9	25.4335s
10	25.7961s
11	26.7655s
12	26.9473s
13	25.3018s
14	24.424s
15	22.4349s
16	24.8827s



Les temps sont bien plus grands que le graphe précédent, on passe de l'ordre du centième de secondes, à centaine de secondes, mais l'évolution du temps semble bien plus efficace, observons l'accélération afin de confirmer :



En effet, on constate une nette accélération, allant jusqu'à $5 \times$ plus vite que la version à 1 thread. De plus, on voit bien ici une évolution ressemblant à celle théorisée par la loi d'Amdahl, l'accélération augmente de manière hyperbolique, puis paraît se stabiliser.

IV Conclusion

On remarque dans cet exercice que la parallélisation a bien plus d'effet sur de grands jeux de données, que sur des petits, nous avons pu observer que, pour cette implantation, les barrières du programme ont un effet moins visible sur les grands jeux de données.

Il semble donc bon de dire que cette parallélisation est bien utile, en particulier pour de grands jeux de données, mais il faudrait voir comment elle se comporterait sur une mise à l'échelle, où les graphes sont bien plus larges qu'une seule ville (malgré la taille de New York), probablement qu'il faudrait adopter une autre approche à cette « simple » parallélisation.

IV.1 Machine utilisée

Tous les temps ont été enregistrés sur un ordinateur portable avec un processeur x86-64 de huit cœurs physiques dont deux virtuels par cœur, cadencé à ~ 3.5 GHz (sur secteur).

IV.2 Données récoltées

Toutes les mesures effectuées sont stockées dans un document *json* récupéré grâce à la sortie standard du programme, et à un script Python, trouvable sur le [GitHub](#).

IV.3 Code

L'entièreté du code se trouve sur le [GitHub](#) dans le fichier Ex6.

IV.4 Notice sur l'utilisation de l'IA générative

L'IA générative n'a pas été utilisée pour cet exercice.